LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

Stack and Queue

Oleh:

ABDURRAZAK **NIM. 2510817110017**

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 1

Laporan Praktikum Algoritma & Struktur Data Modul 2 : Stack and Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : ABDURRAZAK
NIM : 2510817110017

Menyetujui,	Mengetahui,
Asisten Praktikum	Dosen Penanggung Jawab Praktikum
Fadhil Syahdama Mahatma Putra	Andreyan Rizky Baskara, S.Kom., M.Kom
NIM. 2410817210026	NIP. 199307032019031011

DAFTAR ISI

LEMBAR PENGESAHAN i
DAFTAR ISI ii
DAFTAR GAMBAR iii
DAFTAR TABEL iv
SOAL 1 1
A. Jawaban 1
SOAL 2 2
A. Source Code 2
B. Output Program 5
C. Pembahasan 6
SOAL 3 12
A. Source Code 12
B. Output Program 15
C. Pembahasan 16
LINK GITHUB 21

DAFTAR GAMBAR

Gambar 1 Test Push atau Input Data Pada Program Soal Nomor 2..... 5
Gambar 2 Test Pop, Tampilkan, dan Clear Data Pada Program Soal Nomor 2 6
Gambar 3 Test INSERT atau Push Pada Program Soal Nomor 3..... 15
Gambar 4 Test RESET Pada Program Soal Nomor 3..... 15
Gambar 5 Test CETAK Queue Pada Program Soal Nomor 3 15
Gambar 6 Test DELETE Pada Program Soal Nomor 3..... 16

DAFTAR TABEL

Tabel 1 Source Code Soal Nomor 2 2
Tabel 2 Source Code Program Soal Nomor 3 12

SOAL 1

Apa perbedaan Stack dengan Queue?

A. Jawaban

Stack ialah kumpulan elemen data yang disimpan dalam satu jalur linear. Kumpulan elemen data ini hanya boleh dari posisi atas (top) tumpukan. Sedangkan Queue Adalah struktur data linear yang Konsepnya hampir sama dengan stack, perbedaannya adalah operasi penambahan dan penghapusan pada ujung yang berbeda.

Perbedaan dari keduanya juga terdapat pada urutan data yang keluar. Stack meggunakan konsep LIFO (Last In First Out) yang artinya data yang terakhir masuk akan menjadi yang keluar lebih dulu sampai ke data yang paling awal. Sedangkan Queue menggunakan konsep FIFO (First In First Out) yang Dimana urutan data keluarnya dimulai dari yang paling awal masuk sampai data yang terakhir masuk.

Pada operasi yang ada di dalam Stack dan Queue juga ada perbedaan. Dimana pada Stack terdapat operasi Push dan Pop, sedangkan pada Queue ada EnQueue dan DeQueue.

SOAL 2

Cobalah program berikut, running dan analisis hasilnya!

```
int penuh()
{
    if(Tumpuk.atas == max-1)
        return 1;
    else
        return 0;
}

void input (int data)
{
    if (kosong()==1)
    {
        Tumpuk.atas++;
        Tumpuk.data[Tumpuk.atas] = data;
        cout << "Data " << Tumpuk.data[Tumpuk.atas]
             << " Masuk Ke Stack ";
    }
    else if(penuh()==0)
    {
        Tumpuk.atas++;
        Tumpuk.data[Tumpuk.atas] = data;
        cout << "Data "<< Tumpuk.data[Tumpuk.atas]
             << " Masuk Ke Stack ";
    }
    else
        cout << "Tumpukan Penuh";
}
```

```
void hapus()
{
    if(kosong()== 0)
    {
        cout << "Data Teratas Sudah Terambil";
        Tumpuk.atas--;
    }
    else
    cout <<" Data Kosong";
}
void tampil()
{
    if (kosong()== 0)
    {
        for(int i = Tumpuk.atas; i>=0; i--)
        {
            cout <<"\nTumpukan Ke " << i << " = "
            <<Tumpuk.data[i];
        }
    }
    else
    cout << "Tumpukan Kosong";
}
void bersih ()
{
    Tumpuk.atas = -1;
    cout << "Tumpukan Kosong !";
}
```

A. Source Code

Tabel 1 Source Code Soal Nomor 2

```
#include <iostream>
using namespace std;

#define max 5

struct Stack {
    int data[max];
    int atas;
};

Stack Tumpuk;

// cek kosong
int kosong()
{
    if (Tumpuk.atas == -1)
        return 1;
    else
        return 0;
}
```

```

// cek penuh
int penuh()
{
    if (Tumpuk.atas == max - 1)
        return 1;
    else
        return 0;
}

// push
void input(int data)
{
    if (penuh() == 0)
    {
        Tumpuk.atas++;
        Tumpuk.data[Tumpuk.atas] = data;
        cout << "Data " << data << " masuk ke Stack\n";
    }
    else
    {
        cout << "Tumpukan sudah penuh\n";
    }
}

// pop
void hapus()
{
    if (kosong() == 0)
    {
        cout << "Data teratas (" << Tumpuk.data[Tumpuk.atas] << ") dihapus\n";
        Tumpuk.atas--;
    }
    else
    {
        cout << "Tumpukan kosong\n";
    }
}

// tampil
void tampil()
```

```
{
    if (kosong() == 0)
    {
        for (int i = Tumpuk.atas; i >= 0; i--)
        {
            cout << "Tumpukan ke-" << i << " = " << Tumpuk.data[i] << endl;
        }
    }
    else
    {
        cout << "Tumpukan kosong\n";
    }
}

// clear
void bersih()
{
    Tumpuk.atas = -1;
    cout << "Tumpukan dikosongkan\n";
}

int main()
{
    int data;
    int pilihan;

    Tumpuk.atas = -1;

    do
    {
        cout << "\nMenu Stack\n";
        cout << "1. Input Data\n";
        cout << "2. Hapus Data\n";
        cout << "3. Tampil Data\n";
        cout << "4. Bersihkan Stack\n";
        cout << "5. Keluar\n";
        cout << "Pilih Menu: ";
        cin >> pilihan;

        switch (pilihan)
        {
        case 1:
```

```
            cout << "Masukkan Data: ";
            cin >> data;
            input(data);
            break;
        case 2:
            hapus();
            break;
        case 3:
            tampil();
            break;
        case 4:
            bersih();
            break;
        case 5:
            cout << "Keluar dari program.\n";
            break;
        default:
            cout << "Pilihan tidak valid.\n";
        }
    } while (pilihan != 5);

    return 0;
}
```

B. Output Program

```
PS D:\## FIle Kuliah\Semester 2\Praktikum Algo P1\Code\Modul 2> cd 'd:\## FIle Kuliah\Semester 2\Praktikum Algo P1\Code\Modul 2\output'
PS D:\## FIle Kuliah\Semester 2\Praktikum Algo P1\Code\Modul 2\output> & .\'test.exe'

Menu Stack
1. Input Data
2. Hapus Data
3. Tampil Data
4. Bersihkan Stack
5. Keluar
Pilih Menu: 1
Masukkan Data: 67
Data 67 masuk ke Stack

Menu Stack
1. Input Data
2. Hapus Data
3. Tampil Data
4. Bersihkan Stack
5. Keluar
Pilih Menu: 1
Masukkan Data: 11
Data 11 masuk ke Stack
```

Gambar 1 Test Push atau Input Data Pada Program Soal Nomor 2

```
Menu Stack
1. Input Data
2. Hapus Data
3. Tampil Data
4. Bersihkan Stack
5. Keluar
Pilih Menu: 2
Data Teratas Sudah Terambil

Menu Stack
1. Input Data
2. Hapus Data
3. Tampil Data
4. Bersihkan Stack
5. Keluar
Pilih Menu: 3
Tumpukan ke 0 = 67

Menu Stack
1. Input Data
2. Hapus Data
3. Tampil Data
4. Bersihkan Stack
5. Keluar
Pilih Menu: 4
Tumpukan kosong
```

Gambar 2 Test Pop, Tampilkan, dan Clear Data Pada Program Soal Nomor 2

C. Pembahasan

1. Line 1 #include <iostream> digunakan untuk memasukkan library iostream agar program dapat menggunakan fungsi input/output seperti cout dan cin.
2. Line 2 using namespace std; digunakan agar tidak perlu menuliskan std:: saat menggunakan fungsi dari library standar.
3. Line 4 #define max 5 digunakan untuk mendefinisikan konstanta max dengan nilai 5 sebagai kapasitas maksimum stack.
4. Line 6 struct Stack { digunakan untuk mendeklarasikan struktur bernama Stack.
5. Line 7 int data[max]; mendeklarasikan array data dengan ukuran maksimal 5 untuk menyimpan elemen stack.
6. Line 8 int atas; mendeklarasikan variabel atas sebagai penunjuk posisi elemen teratas stack.
7. Line 9 }; menutup deklarasi struktur Stack.
8. Line 11 Stack Tumpuk; mendeklarasikan variabel Tumpuk sebagai stack utama yang digunakan dalam program.
9. Line 14 int kosong() digunakan untuk membuat fungsi pengecekan apakah stack kosong.

10. Line 15 { membuka blok fungsi.
11. Line 16 if (Tumpuk.atas == -1) mengecek apakah posisi atas berada di -1 (kosong).
12. Line 17 return 1; mengembalikan nilai 1 jika stack kosong.
13. Line 18 else kondisi jika stack tidak kosong.
14. Line 19 return 0; mengembalikan nilai 0 jika stack tidak kosong.
15. Line 20 } menutup fungsi kosong.
16. Line 23 int penuh() digunakan untuk membuat fungsi pengecekan apakah stack penuh.
17. Line 24 { membuka blok fungsi.
18. Line 25 if (Tumpuk.atas == max - 1) mengecek apakah stack sudah mencapai kapasitas maksimum.
19. Line 26 return 1; mengembalikan nilai 1 jika stack penuh.
20. Line 27 else kondisi jika stack belum penuh.
21. Line 28 return 0; mengembalikan nilai 0 jika stack belum penuh.
22. Line 29 } menutup fungsi penuh.
23. Line 32 void input(int data) digunakan untuk fungsi push (menambahkan data ke stack).
24. Line 33 { membuka blok fungsi.
25. Line 34 if (penuh() == 0) mengecek apakah stack belum penuh.
26. Line 35 { membuka blok if.
27. Line 36 Tumpuk.atas++; menaikkan posisi atas stack.
28. Line 37 Tumpuk.data[Tumpuk.atas] = data; menyimpan data ke posisi atas stack.
29. Line 38 cout << "Data " << data << " masuk ke Stack\n"; menampilkan pesan bahwa data berhasil ditambahkan.
30. Line 39 } menutup blok if.
31. Line 40 else kondisi jika stack penuh.
32. Line 41 { membuka blok else.

33. Line 42 cout << "Tumpukan sudah penuh\n"; menampilkan pesan bahwa stack penuh.
34. Line 43 } menutup blok else.
35. Line 44 } menutup fungsi input.
36. Line 47 void hapus() digunakan untuk fungsi pop (menghapus data teratas).
37. Line 48 { membuka blok fungsi.
38. Line 49 if (kosong() == 0) mengecek apakah stack tidak kosong.
39. Line 50 { membuka blok if.
40. Line 51 cout << "Data teratas (" << Tumpuk.data[Tumpuk.atas] << ") dihapus\n"; menampilkan data teratas yang akan dihapus.
41. Line 52 Tumpuk.atas--; menurunkan posisi atas stack (menghapus data).
42. Line 53 } menutup blok if.
43. Line 54 else kondisi jika stack kosong.
44. Line 55 { membuka blok else.
45. Line 56 cout << "Tumpukan kosong\n"; menampilkan pesan bahwa stack kosong.
46. Line 57 } menutup blok else.
47. Line 58 } menutup fungsi hapus.
48. Line 61 void tampil() digunakan untuk menampilkan seluruh isi stack.
49. Line 62 { membuka blok fungsi.
50. Line 63 if (kosong() == 0) mengecek apakah stack tidak kosong.
51. Line 64 { membuka blok if.
52. Line 65 for (int i = Tumpuk.atas; i >= 0; i--) melakukan perulangan dari atas ke bawah.
53. Line 66 { membuka blok for.
54. Line 67 cout << "Tumpukan ke-" << i << " = " << Tumpuk.data[i] << endl; menampilkan isi stack dari atas ke bawah.
55. Line 68 } menutup blok for.
56. Line 69 } menutup blok if.

57. Line 70 else kondisi jika stack kosong.
58. Line 71 { membuka blok else.
59. Line 72 cout << "Tumpukan kosong\n"; menampilkan pesan stack kosong.
60. Line 73 } menutup blok else.
61. Line 74 } menutup fungsi tampil.
62. Line 77 void bersih() digunakan untuk mengosongkan stack.
63. Line 78 { membuka blok fungsi.
64. Line 79 Tumpuk.atas = -1; mengatur ulang stack menjadi kosong.
65. Line 80 cout << "Tumpukan dikosongkan\n"; menampilkan pesan bahwa stack telah dibersihkan.
66. Line 81 } menutup fungsi bersih.
67. Line 83 int main() merupakan fungsi utama program.
68. Line 84 { membuka blok main.
69. Line 85 int data; mendeklarasikan variabel untuk menyimpan input data.
70. Line 86 int pilihan; mendeklarasikan variabel untuk menyimpan pilihan menu.
71. Line 88 Tumpuk.atas = -1; menginisialisasi stack sebagai kosong.
72. Line 90 do memulai perulangan menu.
73. Line 91 { membuka blok do.
74. Line 92 cout << "\nMenu Stack\n"; menampilkan judul menu.
75. Line 93 cout << "1. Input Data\n"; menampilkan pilihan menu input data.
76. Line 94 cout << "2. Hapus Data\n"; menampilkan pilihan menu hapus data.
77. Line 95 cout << "3. Tampil Data\n"; menampilkan pilihan menu tampil data.
78. Line 96 cout << "4. Bersihkan Stack\n"; menampilkan pilihan menu bersihkan stack.
79. Line 97 cout << "5. Keluar\n"; menampilkan pilihan keluar.
80. Line 98 cout << "Pilih Menu: "; menampilkan prompt input pilihan.
81. Line 99 cin >> pilihan; menerima input pilihan dari user.
82. Line 101 switch (pilihan) digunakan untuk memilih aksi berdasarkan input user.
83. Line 102 { membuka blok switch.

84. Line 103 case 1: kondisi jika memilih input data.
85. Line 104 cout << "Masukkan Data: "; meminta input data.
86. Line 105 cin >> data; menerima data dari user.
87. Line 106 input(data); memanggil fungsi push.
88. Line 107 break; menghentikan case.
89. Line 108 case 2: kondisi jika memilih hapus data.
90. Line 109 hapus(); memanggil fungsi pop.
91. Line 110 break; menghentikan case.
92. Line 111 case 3: kondisi jika memilih tampil data.
93. Line 112 tampil(); memanggil fungsi tampil.
94. Line 113 break; menghentikan case.
95. Line 114 case 4: kondisi jika memilih bersihkan stack.
96. Line 115 bersih(); memanggil fungsi clear.
97. Line 116 break; menghentikan case.
98. Line 117 case 5: kondisi jika memilih keluar.
99. Line 118 cout << "Keluar dari program.\n"; menampilkan pesan keluar.
100. Line 119 break; menghentikan case.
101. Line 120 default: kondisi jika pilihan tidak valid.
102. Line 121 cout << "Pilihan tidak valid.\n"; menampilkan pesan error.
103. Line 122 } menutup blok switch.
104. Line 123 } while (pilihan != 5); mengulang menu sampai user memilih keluar.
105. Line 125 return 0; mengakhiri program dengan nilai kembali 0.
106. Line 126 } menutup fungsi main dan mengakhiri program.

Analisis :

Pada code yang diberikan soal terdapat beberapa kesalahan dan juga kurangnya beberapa unsur sehingga mengakibatkan code tersebut menjadi eror Ketika saya run.

Setelah menganalisa dan memperbaiki code tersebut, code dapat berjalan dengan baik sesuai dengan tujuan utama dari code tersebut yaitu membuat suatu konsep program LIFO (last In First Out) sederhana.yang memuat beberapa fungsi berupa Push, Pop, Menampilkan Data, dan Clear data. Perubahan dan perbaikan yang saya lakukan ialah sebagai berikut :

1. Define max 5 untuk memberi batasan pada Stack Tumpuk.
2. Menambahkan struct Stack, untuk menyimpan data stack berupa array dan penunjuk posisi teratas (atas), karena sebelumnya belum ada struktur penyimpanan data.
3. Menambahkan fungsi kosong(), untuk mengecek apakah stack dalam kondisi kosong, karena sebelumnya fungsi ini dipanggil tetapi belum dibuat.
4. Menambahkan inisialisasi Tumpuk.atas = -1, agar stack dimulai dalam kondisi kosong dan tidak menggunakan nilai acak.
5. Mengganti penulisan tumpuk menjadi Tumpuk, agar sesuai dengan deklarasi variabel dan menghindari error karena C++ bersifat case-sensitive.
6. Memperbaiki kondisi pada fungsi input(), dari mengecek kosong() menjadi mengecek penuh(), agar sesuai dengan konsep push pada stack.
7. Memperbaiki logika saat stack penuh di fungsi input(), agar tidak tetap menambahkan data ketika stack sudah penuh (mencegah overflow).
8. Memperbaiki format output pada fungsi input(), agar tidak menampilkan data ganda dan lebih mudah dipahami.
9. Menambahkan tampilan data yang dihapus pada fungsi hapus(), agar pengguna mengetahui data mana yang di-pop dari stack.
10. Memperbaiki penggunaan kondisi pada fungsi tampil(), agar hanya menampilkan data ketika stack tidak kosong.
11. Merapikan tampilan output (menambahkan newline dan teks yang lebih jelas), agar program lebih mudah dibaca dan digunakan.

Pada

SOAL 3

Cobalah contoh program berikut, running dan analisis hasilnya!

```
#include<iostream>
#include<conio.h>
#include<stdlib.h>
#define n 10
using namespace std;

void INSERT();
void DELETE();
void CETAKLAYAR();
void Inisialisasi();
void RESET();
int PIL,F,R;
char PILIHAN [1],HURUF;
char Q[n];
int main ( )
{
     Inisialisasi();
     do
     {
          cout<<"QUEUE"<<endl;
          cout<<"==============="<<endl;
          cout<<"1. INSERT"<<endl;
          cout<<"2. DELETE"<<endl;
          cout<<"3. CETAK QUEUE"<<endl;
          cout<<"4. QUIT"<<endl;
          cout<<"PILIHAN : "; cin>>PILIHAN;
          PIL=atoi(PILIHAN);
          switch (PIL)
          {
          case 1:
               INSERT ();
               break;
          case 2:
               DELETE();
               break;
          case 3:
               CETAKLAYAR ();
               break;
          default:
               cout<<"TERIMA KASIH"<<endl;
               break;
          }
          cout<<"press any key to continue"<<endl;
          getch();
          system("cls");
     }
     while (PIL<4);
}
```

A. Source Code

Tabel 2 Source Code Program Soal Nomor 3

```
1   #include <iostream>
2   #include <conio.h>
3   #include <stdlib.h>
4   #define n 10
5   using namespace std;
6
7   int PIL, F, R;
8   char PILIHAN[1], HURUF;
9   char Q[n];
10
11  int full () {
12      if (R == n - 1)
13          return 1;
14      else
15          return 0;
16  }
17
18  int empty () {
19      if (F == -1 || F > R)
20          return 1;
```

```
    else
        return 0;
}

void INSERT() {
    if (full() == 0) {
        cout << "Masukkan Data : ";
        cin >> HURUF;
        if (F == -1)
            F = 0;
        R++;
        Q[R] = HURUF;
    } else {
        cout << "QUEUE PENUH" << endl;
    }
}
void DELETE() {
    if (empty() == 0) {
        cout << "Data yang dihapus : " << Q[F] << endl;
        F++;
    } else {
        cout << "QUEUE KOSONG" << endl;
    }
}

void CETAKLAYAR() {
    if (empty() == 0) {
        cout << "Isi QUEUE : ";
        for (int i = F; i <= R; i++) {
            cout << Q[i] << " ";
        }
        cout << endl;
    } else {
        cout << "QUEUE KOSONG" << endl;
    }
}

void Inisialisasi() {
    F = R = -1;
}
void RESET() {
    F = R = -1;
```

```
63      cout << "QUEUE KOSONG" << endl;
64  }
65
66
67  int main()
68  {
69      Inisialisasi();
70      do
71      {
72          cout << "QUEUE" << endl;
73          cout << "================" << endl;
74          cout << "1. INSERT" << endl;
75          cout << "2. DELETE" << endl;
76          cout << "3. CETAK QUEUE" << endl;
77          cout << "4. RESET" << endl;
78          cout << "5. QUIT" << endl;
79          cout << "PILIHAN : ";
80          cin >> PILIHAN;
81
82          PIL = atoi(PILIHAN);
83
84          switch (PIL)
85          {
86          case 1:
87              INSERT();
88              break;
89          case 2:
90              DELETE();
91              break;
92          case 3:
93              CETAKLAYAR();
94              break;
95          case 4:
96              RESET();
97              break;
98          default:
99              cout << "TERIMA KASIH" << endl;
100             break;
101         }
102
103         cout << "press any key to continue" << endl;
104         getch();
105         system("cls");
```

```

    } while (PIL != 5);
}
```

B. Output Program

```
PS D:\## FIle Kuliah\Semester 2\Praktikum Algo P1\Code\Modul 2\output> cd 'd:\## FIle Kuliah\Semester 2\Praktikum Algo P1\Code\Modul 2\output'
PS D:\## FIle Kuliah\Semester 2\Praktikum Algo P1\Code\Modul 2\output> & .\'Soal 3.exe'
QUEUE
===============
1. INSERT
2. DELETE
3. CETAK QUEUE
4. RESET
5. QUIT
PILIHAN : 1
Masukkan Data : 7
press any key to continue
```

Gambar 3 Test INSERT atau Push Pada Program Soal Nomor 3

```
QUEUE
================
1. INSERT
2. DELETE
3. CETAK QUEUE
4. RESET
5. QUIT
PILIHAN : 4
QUEUE KOSONG
press any key to continue
```

Gambar 4 Test RESET Pada Program Soal Nomor 3

```
QUEUE
================
1. INSERT
2. DELETE
3. CETAK QUEUE
4. RESET
5. QUIT
PILIHAN : 3
Isi QUEUE : 7
press any key to continue
```

Gambar 5 Test CETAK Queue Pada Program Soal Nomor 3

```
QUEUE
===============
1. INSERT
2. DELETE
3. CETAK QUEUE
4. RESET
5. QUIT
PILIHAN : 2
QUEUE KOSONG
press any key to continue
```

Gambar 6 Test DELETE Pada Program Soal Nomor 3

C. Pembahasan

1 Line 1 #include <iostream> digunakan untuk memasukkan library iostream agar dapat menggunakan cout dan cin.

2 Line 2 #include <conio.h> digunakan untuk fungsi getch() agar program menunggu input keyboard.

3 Line 3 #include <stdlib.h> digunakan untuk fungsi atoi() untuk mengubah string menjadi integer.

4 Line 4 #define n 10 mendefinisikan kapasitas maksimum queue sebanyak 10 data.

5 Line 5 using namespace std; digunakan agar tidak perlu menuliskan std:: saat menggunakan fungsi standar.

6 Line 7 int PIL, F, R; mendeklarasikan variabel PIL sebagai pilihan menu, serta F (front) dan R (rear) sebagai penunjuk queue.

7 Line 8 char PILIHAN[1], HURUF; mendeklarasikan array karakter untuk input pilihan dan variabel huruf untuk data queue.

8 Line 9 char Q[n]; mendeklarasikan array queue dengan kapasitas n.

9 Line 11 int full () { mendeklarasikan fungsi untuk mengecek apakah queue penuh.

10 Line 12 if (R == n - 1) mengecek apakah posisi rear sudah di indeks terakhir.

11 Line 13 return 1; mengembalikan nilai 1 jika penuh.

12 Line 14 else kondisi jika tidak penuh.

13 Line 15 return 0; mengembalikan nilai 0 jika tidak penuh.

14 Line 16 } menutup fungsi full.

15 Line 18 int empty () { mendeklarasikan fungsi untuk mengecek apakah queue kosong.

16 Line 19 if (F == -1 || F > R) mengecek apakah queue kosong.

17 Line 20 return 1; mengembalikan nilai 1 jika kosong.

18 Line 21 else kondisi jika tidak kosong.

19 Line 22 return 0; mengembalikan nilai 0 jika tidak kosong.

20 Line 23 } menutup fungsi empty.

21 Line 25 void INSERT() { mendeklarasikan fungsi untuk menambahkan data ke queue (enqueue).

22 Line 26 if (full() == 0) { mengecek apakah queue belum penuh.

23 Line 27 cout << "Masukkan Data : "; menampilkan prompt input.

24 Line 28 cin >> HURUF; menerima input karakter dari user.

25 Line 29 if (F == -1) mengecek jika queue masih kosong.

26 Line 30 F = 0; mengatur posisi front ke indeks awal.

27 Line 31 R++; menaikkan posisi rear.

28 Line 32 Q[R] = HURUF; menyimpan data ke queue.

29 Line 33 } else { kondisi jika queue penuh.

30 Line 34 cout << "QUEUE PENUH" << endl; menampilkan pesan penuh.

31 Line 35 } menutup fungsi INSERT.

32 Line 36 void DELETE() { mendeklarasikan fungsi untuk menghapus data dari queue (dequeue).

33 Line 37 if (empty() == 0) { mengecek apakah queue tidak kosong.

34 Line 38 cout << "Data yang dihapus : " << Q[F] << endl; menampilkan data yang dihapus.

35 Line 39 F++; memajukan posisi front.

36 Line 40 } else { kondisi jika queue kosong.

37 Line 41 cout << "QUEUE KOSONG" << endl; menampilkan pesan kosong.

38 Line 42 } menutup fungsi DELETE.

39 Line 44 void CETAKLAYAR() { mendeklarasikan fungsi untuk menampilkan isi queue.

40 Line 45 if (empty() == 0) { mengecek apakah queue tidak kosong.

41 Line 46 cout << "Isi QUEUE : "; menampilkan label isi queue.

42 Line 47 for (int i = F; i <= R; i++) { melakukan perulangan dari front ke rear.

43 Line 48 cout << Q[i] << " "; menampilkan setiap elemen queue.

44 Line 49 } menutup perulangan.

45 Line 50 cout << endl; menambahkan baris baru.

46 Line 51 } else { kondisi jika queue kosong.

47 Line 52 cout << "QUEUE KOSONG" << endl; menampilkan pesan kosong.

48 Line 53 } menutup fungsi CETAKLAYAR.

49 Line 55 void Inisialisasi() { mendeklarasikan fungsi untuk inisialisasi queue.

50 Line 56 F = R = -1; mengatur front dan rear ke -1 (kosong).

51 Line 57 } menutup fungsi Inisialisasi.

52 Line 58 void RESET() { mendeklarasikan fungsi untuk mengosongkan queue.

53 Line 59 F = R = -1; mengatur ulang queue menjadi kosong.

54 Line 60 cout << "QUEUE KOSONG" << endl; menampilkan pesan reset.

55 Line 61 } menutup fungsi RESET.

56 Line 64 int main() merupakan fungsi utama program.

57 Line 65 { membuka blok main.

58 Line 66 Inisialisasi(); memanggil fungsi untuk mengosongkan queue di awal.

59 Line 67 do memulai perulangan menu.

60 Line 68 { membuka blok do.

61 Line 69 cout << "QUEUE" << endl; menampilkan judul program.

62 Line 70 cout << "==========" << endl; menampilkan garis pembatas.

63 Line 71 cout << "1. INSERT" << endl; menampilkan menu insert.

64 Line 72 cout << "2. DELETE" << endl; menampilkan menu delete.

65 Line 73 cout << "3. CETAK QUEUE" << endl; menampilkan menu cetak.

66 Line 74 cout << "4. RESET" << endl; menampilkan menu reset.

Line 75 cout << "5. QUIT" << endl; menampilkan menu keluar.

Line 76 cout << "PILIHAN : "; menampilkan input pilihan.

Line 77 cin >> PILIHAN; menerima input pilihan dalam bentuk string.

Line 79 PIL = atoi(PILIHAN); mengubah input string menjadi integer.

Line 81 switch (PIL) digunakan untuk memilih aksi berdasarkan input.

Line 82 { membuka blok switch.

Line 83 case 1: kondisi jika memilih insert.

Line 84 INSERT(); memanggil fungsi enqueue.

Line 85 break; menghentikan case.

Line 86 case 2: kondisi jika memilih delete.

Line 87 DELETE(); memanggil fungsi dequeue.

Line 88 break; menghentikan case.

Line 89 case 3: kondisi jika memilih cetak.

Line 90 CETAKLAYAR(); menampilkan isi queue.

Line 91 break; menghentikan case.

Line 92 case 4: kondisi jika memilih reset.

Line 93 RESET(); mengosongkan queue.

Line 94 break; menghentikan case.

Line 95 default: kondisi selain pilihan 1–4.

Line 96 cout << "TERIMA KASIH" << endl; menampilkan pesan keluar.

Line 97 break; menghentikan case.

Line 98 } menutup blok switch.

Line 100 cout << "press any key to continue" << endl; menampilkan pesan lanjut.

Line 101 getch(); menunggu input tombol dari user.

Line 102 system("cls"); membersihkan layar console.

Line 104 } while (PIL != 5); mengulang menu sampai user memilih keluar.

Line 105 } menutup fungsi main dan mengakhiri program.

Aalisis :

Pada bagian soal ini diminta untuk melengkapi barisan code yang telah disediakan dengan penambahan code hingga selesai sebagai berikut :

1 Menambahkan menu RESET pada tampilan utama, supaya pengguna dapat mengosongkan seluruh isi queue tanpa harus keluar dari program.
2 Menambahkan fungsi RESET(), penambahan ini bertujuan untuk mengembalikan nilai front (F) dan rear (R) menjadi -1 sehingga queue kembali ke kondisi kosong.
3 Menambahkan case RESET pada struktur switch, supaya menu RESET yang dipilih oleh pengguna dapat menjalankan fungsi RESET() dengan benar.
4 Mengubah posisi menu QUIT dari pilihan ke-4 menjadi ke-5, supaya memberi ruang untuk penambahan fitur RESET tanpa mengganggu fungsi lainnya.
5 Mengubah kondisi perulangan dari while (PIL != 4) menjadi while (PIL != 5), perubahan ini bertujuan agar program berhenti sesuai dengan nomor menu QUIT yang baru.
6 Menambahkan output pada fungsi DELETE(), supaya program menampilkan data yang dihapus dari queue sehingga lebih informatif bagi pengguna.
7 Menyempurnakan fungsi CETAKLAYAR() dengan perulangan dari F hingga R, supaya seluruh isi queue dapat ditampilkan secara berurutan dari depan ke belakang.
8 Menambahkan inisialisasi queue melalui fungsi Inisialisasi() di awal program, supaya queue dimulai dalam kondisi kosong dan terhindar dari nilai acak.
9 Menggunakan fungsi full() dan empty() secara konsisten pada INSERT() dan DELETE(), supaya proses penambahan dan penghapusan data berjalan sesuai dengan konsep queue dan terhindar dari error seperti overflow dan underflow.

LINK GITHUB

Rzkkk16/Praktikum-Algoritma-P1